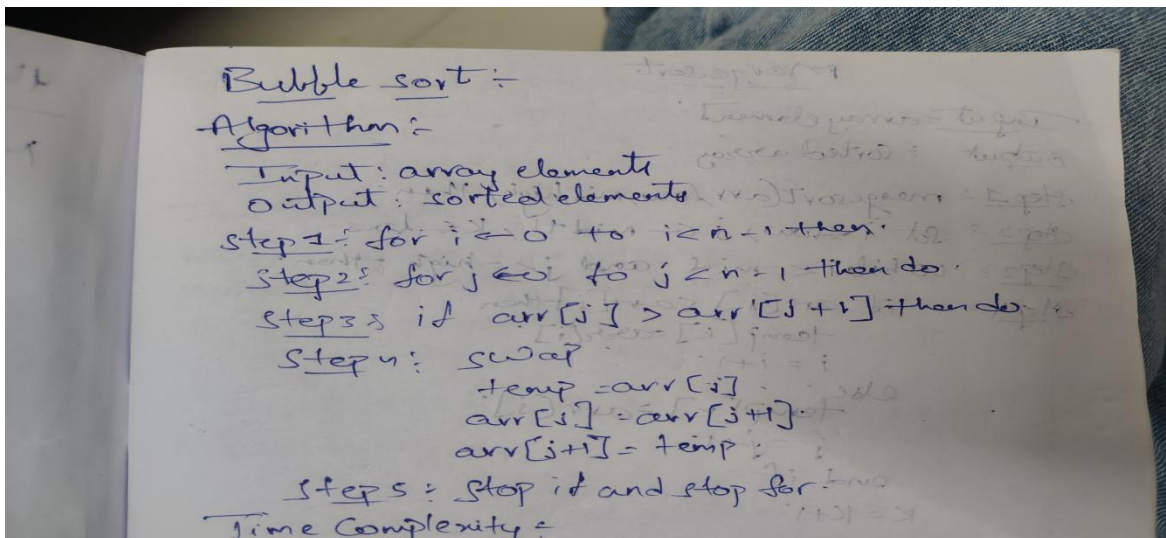


PRACTICAL-1

AIM: Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

BUBBLE SORT

ALGORITHM:



CODE:

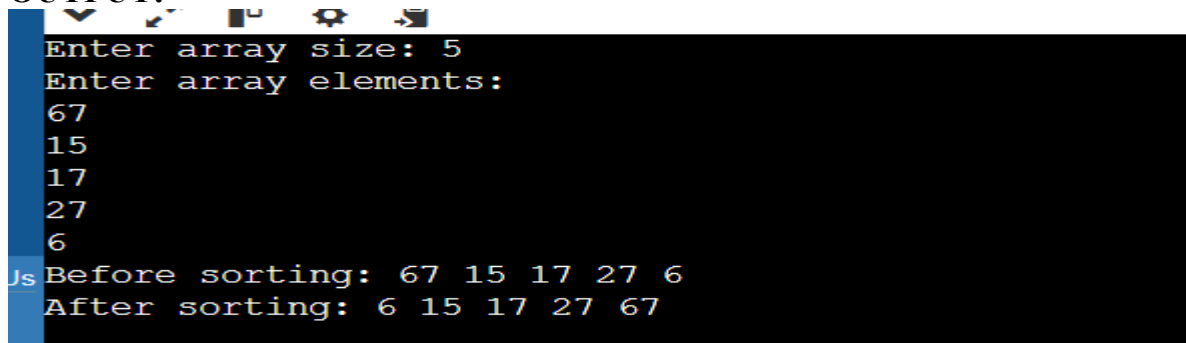
```
#include <iostream>
using namespace std;
class bubblesort
{
public:
    int size, arr[100], i;
    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;
        cout << "Enter array elements:\n";
        for (i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }
    void printarray()
    {
        for (i = 0; i < size; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
}
```

```
void bubblelogic()
{
    int j, temp;
    for (i = 0; i < size - 1; i++)
    {
        for (j = 0; j < size - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

};

int main()
{
    bubblesort bs;
    bs.arrayinput();
    cout << "Before sorting: ";
    bs.printarray();
    bs.bubblelogic();
    cout << "After sorting: ";
    bs.printarray();
    return 0;
}
```

OUTPUT:



The screenshot shows a terminal window with a dark background. The user enters the array size as 5 and the elements as 67, 15, 17, 27, and 6. The program then displays the array before sorting (67 15 17 27 6) and after sorting (6 15 17 27 67). The output is displayed in a monospaced font with a blue cursor line.

```
Enter array size: 5
Enter array elements:
67
15
17
27
6
Before sorting: 67 15 17 27 6
After sorting: 6 15 17 27 67
```

TIME COMPLEXITY:

Time Complexity =

```

for (int i=0; i<n-1; i++) {
    for (j=0; j<n-1-i; j++) {
        if (arr[j] > arr[j+1]) {
            temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
}

```

$T = (n-1) \times (n-1) \times 1 \times 1 \times 1 \times 1$
 $= (n-1)^2$
 $= n^2 - 2n + 1$
 $= n^2 - 2n$
 $T(n) = O(n^2)$

Bubble Sort:

Array = [5, 3, 4, 1, 2]

Time Complexity:

Best Case:

Already sorted array: [1, 2, 3, 4, 5]

compare adjacent elements

- 1 vs 2 → 1 comparison
- 2 vs 3 → 1 comparison
- 3 vs 4 → 1 comparison
- 4 vs 5 → 1 comparison

Best Case = $O(n)$

Worst Case:

Reverse sorted: [5, 4, 3, 2, 1]

No. of comparisons

First pass → 4
 Second pass → 3
 Third pass → 2
 Fourth pass → 1

Formula: $\frac{n(n-1)}{2}$

$n=5 \Rightarrow \frac{5(4)}{2} = 10$

Worst case = $O(n^2)$

Average Case: Take any random array.

Possible comparisons: $\frac{n(n-1)}{2}$

$n=5 \Rightarrow \frac{5(4)}{2} = 10$

Average case = $O(n^2)$

SELECTION SORT

ALGORITHM:

Algorithm:

Step 1: For $i \leftarrow 0$ to $\text{size} - 1$, do.

Step 2: $\text{min} \leftarrow i$, then.

Step 3: For $j \leftarrow i + 1$ to n , then do.

Step 4: IF $\text{arr}[j] < \text{arr}[\text{min}]$, do.

Step 5: $\text{min} \leftarrow j$

Step 6: end if, end for.

Step 7: IF $\text{min} \neq i$ then do.

Step 8: Swap

Step 9: $\text{temp} \leftarrow \text{arr}[i]$ Step 9: end if

Step 9: $\text{arr}[i] \leftarrow \text{arr}[\text{min}]$ Step 10: end for

$\text{arr}[\text{min}] \leftarrow \text{temp}$

Step 8: Swap

Step 9: $\text{temp} \leftarrow \text{arr}[i]$

$\text{arr}[i] \leftarrow \text{arr}[\text{min}]$

$\text{arr}[\text{min}] \leftarrow \text{temp}$

Step 9: end if

Step 10: end for.

CODE:

```
#include <iostream>
using namespace std;

class SelectionSort
{
public:
    int size, arr[100], i, j, temp;

    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;

        cout << "Enter array elements: ";
        for (i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }

    void printarray()
    {
        for (i = 0; i < size; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

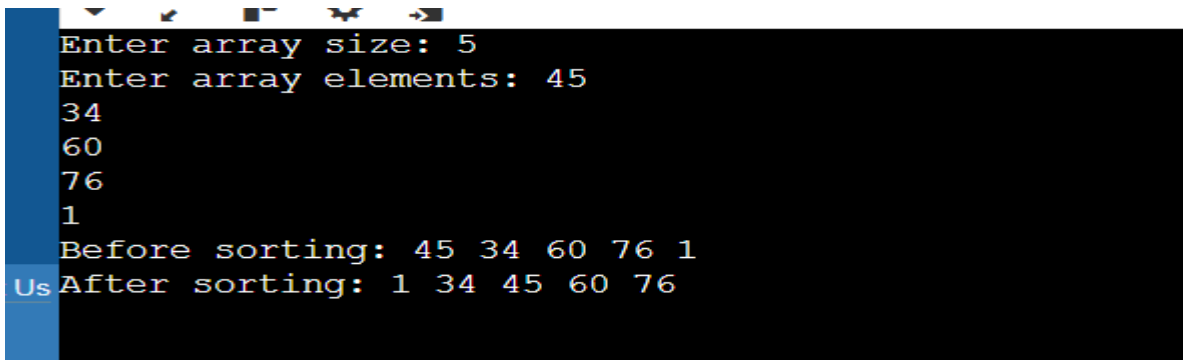
    void selectionsortlogic()
    {
        for (i = 0; i < size - 1; i++)
        {
            int min = i;

            for (j = i + 1; j < size; j++)
            {
                if (arr[j] < arr[min])
                {
                    min = j;
                }
            }

            if (min != i)
            {
                temp = arr[i];
                arr[i] = arr[min];
                arr[min] = temp;
            }
        }
    }
}
```

```
};  
  
int main()  
{  
    SelectionSort ss;  
  
    ss.arrayinput();  
  
    cout << "Before sorting: ";  
    ss.printarray();  
  
    ss.selectionsortlogic();  
  
    cout << "After sorting: ";  
    ss.printarray();  
  
    return 0;  
}
```

OUTPUT:



```
Enter array size: 5  
Enter array elements: 45  
34  
60  
76  
1  
Before sorting: 45 34 60 76 1  
Us After sorting: 1 34 45 60 76
```

TIME COMPLEXITY:

Selection Sort:

```

for (i=0; i < size-1; i++) → n-1
    int min=i; → 1
    for (j=i+1; j < n; j++) → n
        if (arr[j] < arr[min]) {
            min=j; → 1
        }
    if (min != i) {
        temp = arr[i]; → 1
        arr[i] = arr[min]; → 1
        arr[min] = temp; → 1
    }
}

```

Algorithm:

$$T.C = (n-1)(n) \times (1 \times 1 \times 1 \times 1)$$

$$= (n-1)(n)$$

$$= n^2 - n$$

$$T.C = O(n^2)$$

$$T(n) = O(n^2)$$

Selection Sort:

Array: [5, 5, 4, 1, 2].

Comparisons: 4.

Swap 1 with 5.

[1, 3, 4, 5, 2].

Pass 2:

Remaining [3, 4, 5, 2].

Comparisons: 3.

Pass 3:

Remaining [4, 5, 3].

Comparisons: 2.

Pass 4:

Remaining [5, 4].

Comparison: 1.

Total = 4 + 3 + 2 + 1 = 10.

Formula: $\frac{n(n-1)}{2}$

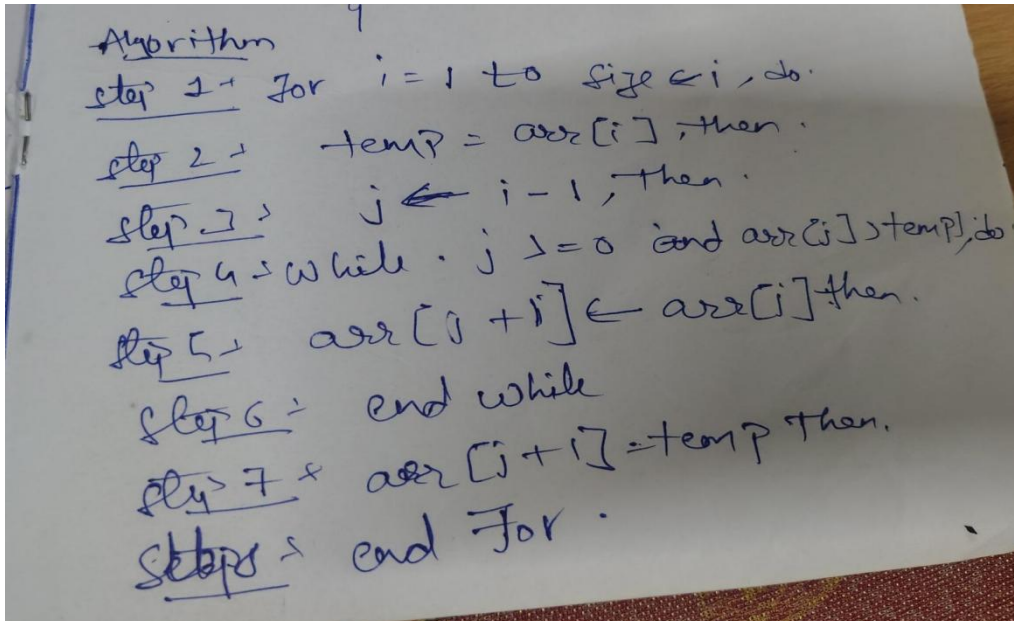
$n = 5 \Rightarrow \frac{5(4)}{2} = 10$.

For every case it is same.

- Best Case = $O(n^2)$
- Average Case = $O(n^2)$
- Worst Case = $O(n^2)$

INSERTION SORT

ALGORITHM:



CODE:

```
#include <iostream>
using namespace std;

class InsertionSort
{
public:
    int size, arr[100], i, j, temp;

    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;

        cout << "Enter array elements: ";
        for (i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }

    void printarray()
    {
        for (i = 0; i < size; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
};
```

```
}  
  
void insertionsortlogic()  
{  
    for (i = 1; i < size; i++)  
    {  
        temp = arr[i];  
        j = i - 1;  
  
        while (j >= 0 && arr[j] > temp)  
        {  
            arr[j + 1] = arr[j];  
            j--;  
        }  
  
        arr[j + 1] = temp;  
    }  
}  
};  
  
int main()  
{  
    InsertionSort is;  
  
    is.arrayinput();  
  
    cout << "Before sorting: ";  
    is.printarray();  
  
    is.insertionsortlogic();  
  
    cout << "After sorting: ";  
    is.printarray();  
  
    return 0;  
}
```

OUTPUT:

```
Enter array size: 5  
Enter array elements: 45  
33  
9  
0  
6  
Before sorting: 45 33 9 0 6  
After sorting: 0 6 9 33 45
```

TIME COMPLEXITY:

Insertion sort
 for $i = 1; i < \text{size}; i++ \} \leftarrow n$
 $\text{temp} = \text{arr}[i]; \rightarrow 1$
 $j = i - 1; \rightarrow 1$
 while $(j > 0 \ \& \ \text{arr}[j] > \text{temp}) \leftarrow n$
 $\text{arr}[j+1] = \text{arr}[j]; \rightarrow 1$
 $j--; \rightarrow 1$
 $\text{arr}[j+1] = \text{temp}; \rightarrow 1$
 T.C. = $(n)(n)(1 \times 1)$
 $= n^2$
 $T(n) = O(n^2)$

Insertion sort
 Array: $[5, 8, 4, 1, 2]$
Best case - Already sorted
 $[1, 2, 3, 4, 5]$
 Check each element with previous element
 • $2 \rightarrow 1$ Comparing
 • $3 \rightarrow 2$
 • $4 \rightarrow 3$
 • $5 \rightarrow 4$
 Total: $n - 1 = 4$
 Best case = $O(n)$
Worst case - Reverse Sorted
 $[5, 4, 3, 2, 1]$
 every new element has to move to beginning
Comparisons:
 for $4 \rightarrow 1$
 for $3 \rightarrow 2$
 for $2 \rightarrow 1$
 for $1 \rightarrow 0$
 Total: $4 + 3 + 2 + 1 = 10$
Formula: $\frac{n(n-1)}{2} = (n) \cdot \frac{n-1}{2}$
 $n = 5 \Rightarrow \frac{5(4)}{2} = 10$
 Worst case = $O(n^2)$
Average case: Random Arrangement we can take,
 each element moves through about half of previous element
Approximate comparisons $\frac{n(n-1)}{4}$
 $n = 5 \Rightarrow \frac{5(4)}{4} = 5$
 Average case = $O(n^2)$

MERGE SORT

ALGORITHM:

Mergesort

Input: array element
Output: sorted array

Step 1: mergesort(arr, low, mid, high) then

Step 2: set $i \leftarrow \text{low}$, $j \leftarrow \text{mid} + 1$, $k \leftarrow \text{low}$

Step 3: while $i \leq \text{mid}$ and $j \leq \text{high}$ then

Step 4: if $\text{arr}[i] < \text{arr}[j]$ then
temp[k] = arr[i]
 $i = i + 1$
else
temp[k] = arr[j]
 $j = j + 1$
and if
 $k = k + 1$
end if

Step 5: while $j \leq \text{high}$ then
temp[k] = arr[j]
 $k = k + 1$
end while

Step 6: while $i \leq \text{mid}$ then
temp[k] = arr[i]
 $i = i + 1$
 $k = k + 1$
end while

Step 7: for $i \leftarrow \text{low}$, $k \leftarrow \text{low}$ to $i \leq \text{high}$ then
arr[i] = temp[k]

Step 8: MERGESORT(arr, low, high) then do

Step 9: if $\text{low} < \text{high}$ then do

$\text{mid} = (\text{low} + \text{high}) / 2$

MERGE-SORT(arr, low, mid)

MERGE-SORT(arr, mid+1, high)

end if MERGE(arr, low, mid, high)

CODE:

```
#include <iostream>
using namespace std;

class mergesort
{
public:
    int size, arr[100], i;

    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;

        cout << "Enter array elements:\n";
        for (i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }

    void printarray()
    {
        for (i = 0; i < size; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    void merge(int low, int mid, int high)
    {
        int temp[100];
        int i = low, j = mid + 1, k = low;

        while (i <= mid && j <= high)
        {
            if (arr[i] <= arr[j])
            {
                temp[k] = arr[i];
                i++;
            }
            else
            {
                temp[k] = arr[j];
                j++;
            }
            k++;
        }

        while (i <= mid)
        {
```

```
temp[k] = arr[i];
i++;
k++;
}

while (j <= high)
{
temp[k] = arr[j];
j++;
k++;
}

for (i = low; i <= high; i++)
{
arr[i] = temp[i];
}
}

void mergellogic(int low, int high)
{
if (low < high)
{
int mid = (low + high) / 2;

mergellogic(low, mid);
mergellogic(mid + 1, high);

merge(low, mid, high);
}
}
};

int main()
{
mergesort ms;

ms.arrayinput();

cout << "Before sorting: ";
ms.printarray();

ms.mergellogic(0, ms.size - 1);

cout << "After sorting: ";
ms.printarray();

return 0;
}
```

OUTPUT:

```
Enter array size: 6
Enter array elements:
78
34
5
7
54
66
Before sorting: 78 34 5 7 54 66
After sorting: 5 7 34 54 66 78
```

TIME COMPLEXITY:

Time Complexity :-

Merge sort(arr, low, mid) $\rightarrow n/2$
 merge sort(arr, mid+1, high) $\rightarrow n/2$
 merge (arr, low, mid, high) $\rightarrow n$
 $T(n) = T(n/2) + T(n/2) + n$
 $= 2T(n/2) + n$

Master method :-

$T(n) = aT(n/b) + f(n)$
 $a = 2 \quad b = 2 \quad f(n) = n \quad p = 0 \quad k = 1$
 $n \log_b a = n \log_2 2 = n$
 $f(n) = n$
 $n \log_b a = f(n) \Rightarrow n \log_2 2 = n$
 Case 2: $p > -1$
 $T(n) = O(n \log^{p+1} n)$
 $T(n) = O(n \log n)$

For standard merge sort, the recursion is essentially the same regardless of the initial arrangement.

Best Case $= O(n \log 1)$
 Worst Case $= O(n \log n)$
 Average Case $= O(n \log n)$

QUICK SORT

ALGORITHM:

Quick Sort

Inputs: array elements
Output: sorted array

Step 1 → Partition(arr, low, high) then

Step 2 → Set pivot = arr[high]
Set i = low

Step 3 → for j = low to high - 1 then

Step 4 → if arr[j] < pivot then
i ← i + 1
swap arr[i] and arr[j]
end if
end for

Step 5 → swap arr[i + 1] and arr[high]

Step 6 → Return i + 1

Step 7 → QuickSort(arr, low, high) then

Step 8 → if low < high then do
Pivot index = Partition(arr, low, high)
QuickSort(arr, low, Pivot index - 1)
QuickSort(arr, Pivot index + 1, high)
end if

CODE:

```
#include <iostream>
#include <utility>
using namespace std;

class QuickSort
{
public:
    int size, arr[100];

    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;

        cout << "Enter array elements: ";
        for (int i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }

    void printarray()
    {
        for (int i = 0; i < size; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    int partition(int lb, int ub)
    {
        int pivot = arr[lb];
        int start = lb;
        int end = ub;

        while (start < end)
        {
            while (start <= ub && arr[start] <= pivot)
                start++;

            while (arr[end] > pivot)
                end--;

            if (start < end)
                swap(arr[start], arr[end]);
        }
    }
}
```

```
swap(arr[lb], arr[end]);
return end;
}

void quicksortlogic(int lb, int ub)
{
    if (lb < ub)
    {
        int loc = partition(lb, ub);
        quicksortlogic(lb, loc - 1);
        quicksortlogic(loc + 1, ub);
    }
}

void sort()
{
    quicksortlogic(0, size - 1);
}
};

int main()
{
    QuickSort qs;

    qs.arrayinput();

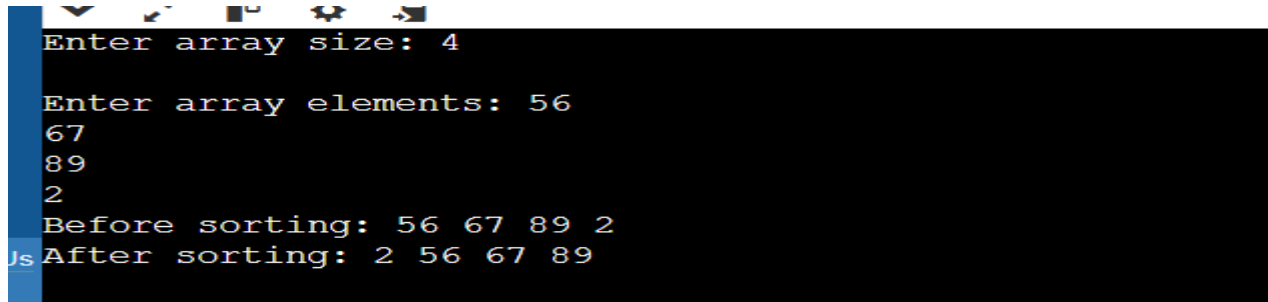
    cout << "Before sorting: ";
    qs.printarray();

    qs.sort();

    cout << "After sorting: ";
    qs.printarray();

    return 0;
}
```

OUTPUT:



```
Enter array size: 4
Enter array elements: 56
67
89
2
Before sorting: 56 67 89 2
After sorting: 2 56 67 89
```

TIME COMPLEXITY:

Time Complexity
 quicksort (case, low, pivot index-1) $\rightarrow n/2$
 quicksort (case, pivot index+1, high) $\rightarrow n/2$
 Partition (case, low, high); $\rightarrow n$

$$T(n) = T(n/2) + T(n/2) + n$$

$$= 2T(n/2) + n$$

Master method
 $T(n) = aT(n/b) + f(n)$
 $a = 2, b = 2, f(n) = n, P = 0, k = 1$
 $n \log_b a = n \log_2 2 = n$
 $\log_b a = k$
 Case $\rightarrow P > -1$
 $T(n) = O(n \log^P n)$
 $\boxed{T(n) = O(n \log n)}$ - Best case

Quicksort
 - Average case: In average case pivot doesn't always divide perfectly equal, but the average partitions are reasonably balanced.
 Average case = $O(n \log n)$
 - Worst case: quicksort (left) $\rightarrow T(n-1)$
 Partition $\rightarrow n$
 quicksort (right) $\rightarrow T(0)$
 $T(n) = T(n-1) + n$
 $T(n) = n + (n-1) + (n-2) + \dots + 1$
 $= n(n+1)/2$
 $= \frac{n^2 + n}{2}$
 $\boxed{T(n) = O(n^2)}$

CONCLUSION:

The sorting algorithms Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort were successfully implemented and analyzed. Each algorithm has a different way of sorting elements and different time requirements. Merge Sort and Quick Sort are generally faster for large data, while Bubble, Selection, and Insertion Sort are simple and easy to understand.